

C++ 프로그래밍

Junho Kim

Kookmin University

Course Instructor

- 김준호 (Junho Kim)
- Research Intrests
 - Computer Graphics
 - Computer Vision
 - Mixed Reality
- Contact Info.
 - ✉ : junho@kookmin.ac.kr
 - ☎ : 02-910-4826



코드트 헬퍼 (근무 위치: 자율주행스튜디오)

헬퍼 제도: 근무시간에 helper room에 상주하며, 방문하는 수강생들에 대해 코딩 실습 지원 방문하는 수강생들에 대해 다양한 질의 대응 및 애로 사항 해결

- 박건민 (소프트웨어학부 4학년)

- 월요일: 10:30 - 13:00
- 수요일: 10:30 - 13:00
- 금요일: 12:00 - 15:00

- 장근서 (인공지능학부 4학년)

- 월요일: 14:00 ~ 15:00 (1시간)
- 화요일: 13:30 ~ 17:30 (4시간)
- 목요일: 13:30 ~ 16:30 (3시간)

교과목 개요

- 시스템 프로그래밍에 사용되는 C/C++ 언어를 배워 프로그램의 low-level 작동 메카니즘을 이해하고 python/java 언어에서 배운 객체 지향 프로그램 기법을 C++언어에서 사용하는 법을 배운다.
- 프로그램의 low-level 작동 메카니즘을 이해하기 위하여 개발 환경은 Linux에서 text editor와 gcc comiler, gdb debugger를 사용한다.
- STL 의 활용법을 배워 복잡한 자료구조를 쉽게 이용하도록 한다.

수업목표

1. C 프로그래밍 언어를 학습하여 프로그램이 컴퓨터 시스템에서 수행되는 메커니즘을 설명할 수 있다.
2. C++ 프로그래밍 언어에서 객체지향 개념을 활용하여 프로그램에 적용할 수 있다.
3. C/C++ 프로그램 개발에 STL 을 활용할 수 있다.

평가방법 (tentative)

- 중간고사: 25%
- 기말고사: 25%
- 프로젝트/과제물: 10%
- 실습: 20%
- 수업참여도: 10%

중간고사/기말고사

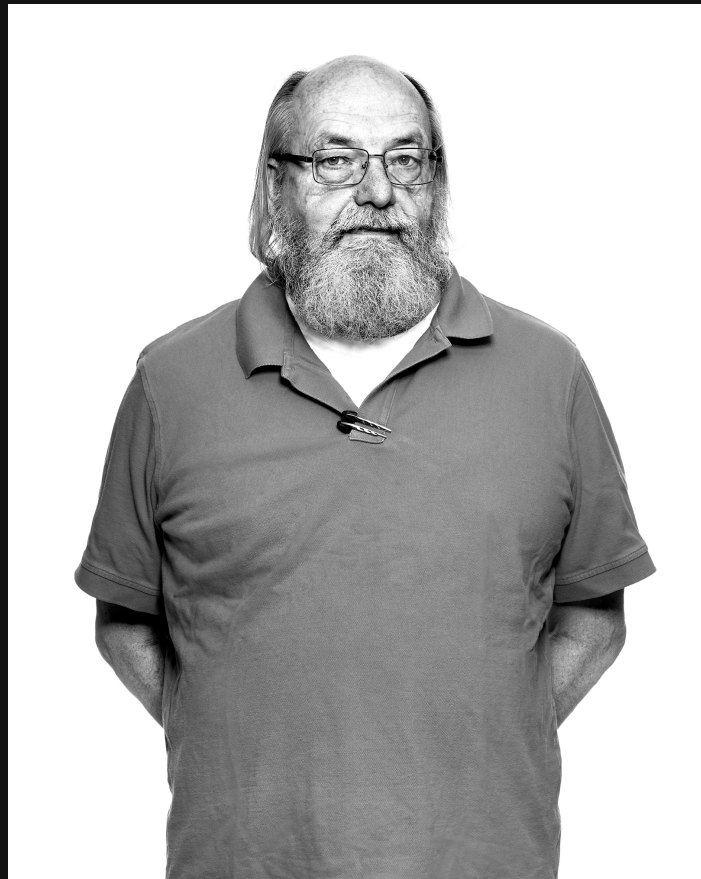
- 시험일자: 추후 재공지

C 언어의 탄생

Bell Labs와 UNIX

Bell Labs와 UNIX

- 1969년 — Ken Thompson, AT&T Bell Labs에서 UNIX 운영체제 개발 시작
- 당시 OS는 어셈블리어로 작성 → 이식성 없음, 유지보수 어려움



C 언어의 탄생

- 1972년 — Dennis Ritchie, B 언어를 발전시켜 C 언어 개발
- UNIX를 C로 재작성 → "시스템을 직접 제어할 수 있는 고 수준 언어"의 시작



C 언어 설계 철학

💡 C 언어: "하드웨어에 가깝지만, 사람이 읽을 수 있는 언어"를 목표로 설계

```
1 1969  UNIX 개발 시작 (Assembly)
2  |
3 1972  Dennis Ritchie → C 언어 탄생
4  |
5 1978  "The C Programming Language" 출판 (K&R C)
6  |
7 1989  ANSI C (C89) 표준화
```

C에서 C++로

"C with Classes"

Bjarne Stroustrup

- 1979년 — 덴마크 출신 컴퓨터 과학자, Bell Labs에서 연구
- 박사 논문 작업 중 Simula(최초의 OOP 언어)의 클래스 개념에 주목
- C의 성능 + Simula의 OOP → "C with Classes" 개발 시작



C++로의 진화

```
1 1979 "C with Classes" 개발 시작 (Bjarne Stroustrup)
2   |
3 1983 정식 명칭 "C++" 으로 변경
4     ( ++ 는 C의 증가 연산자, "C를 한 단계 발전시킨 언어" )
5   |
6 1985 "The C++ Programming Language" 초판 출판
7   |
8 1998 C++98 - 최초의 ISO 국제 표준
```

💡 ++ 는 증가 연산자 — *"C보다 한 단계 나아간 언어"*라는 의미를 담고 있습니다.

C++ 표준의 역사

```
1  C++98  ─ 최초 ISO 표준. STL, 템플릿 도입
2  |
3  C++03  ─ C++98 버그 수정판
4  |
5  C++11  ─ ★ 대격변. auto, lambda, 스마트 포인터,
6  |         move semantics, range-based for, nullptr
7  |
8  C++14  ─ C++11 개선 (generic lambda 등)
9  |
10 C++17  ─ structured bindings, if constexpr,
11 |        std::optional, std::variant
12 |
13 C++20  ─ Concepts, Ranges, Coroutines, Modules
14 |
15 C++23  ─ std::print, std::expected 등 추가
```

이 강의의 기준: C++11, C++17

- 현업에서 가장 널리 사용되는 표준
- Modern C++ 문법을 충분히 활용 가능

💡 C++11을 기점으로 "Modern C++"와 "Legacy C++"로 나뉩니다.

C와 C++의 차이점

C는 C++에 포함되는가?

"C++는 C의 상위 집합(superset)에 가깝지만, 완전히 동일하지는 않습니다."

항목	C	C++
패러다임	절차적(Procedural)	절차적 + OOP + 제네릭
클래스 / 객체	✗	✓
함수 오버로딩	✗	✓
참조(Reference)	✗	✓
표준 라이브러리	제한적 (stdio.h 등)	방대한 STL
메모리 관리	<code>malloc</code> / <code>free</code>	<code>new</code> / <code>delete</code> + 스마트 포인터
타입 안전성	낮음	높음

예시 비교

```
1  /* C 스타일 */
2  #include <stdio.h>
3  int main() {
4      printf("Hello, C!\n");
5      return 0;
6  }
```

```
1  // C++ 스타일
2  #include <iostream>
3  int main() {
4      std::cout << "Hello, C++!" << std::endl;
5      return 0;
6  }
```

C++의 설계 철학

"Zero-overhead Abstraction"

"What you don't use, you don't pay for." — Bjarne Stroustrup

- 추상화(클래스, 템플릿 등)를 사용해도 **성능 손실이 없어야 한다**
- 직접 구현했을 때와 동일하거나 더 나은 성능을 목표로 함

3가지 핵심 철학

철학	설명
Zero-overhead	쓰지 않는 기능에 비용을 지불하지 않는다
표현력	복잡한 개념을 간결하게 표현할 수 있어야 한다
하드웨어와의 근접성	하드웨어를 직접 제어할 수 있어야 한다

💡 Python, Java가 편의성을 위해 성능을 일부 희생하는 반면,
C++는 **성능을 포기하지 않는 추상화**를 지향합니다.

C++가 사용되는 분야 & 현업 사례

주요 사용 분야

분야	대표 사례
 게임 엔진	Unreal Engine, id Tech (Doom), CryEngine
 브라우저 / JS 엔진	Chrome (V8), Firefox (SpiderMonkey)
 운영체제	Windows 커널 일부, macOS 일부
 금융 / HFT	초저지연 거래 시스템
 AI / ML 프레임워크	TensorFlow, PyTorch (C++ 백엔드)
 임베디드 / IoT	자동차 ECU, 드론 펌웨어
 데이터베이스	MySQL, MongoDB, RocksDB

💡 Python으로 작성된 TensorFlow도 내부 연산은 C++로 구현되어 있습니다.

다른 언어와의 포지셔닝

언어별 포지셔닝 요약

언어	강점	C++ 대비
Python	생산성, 생태계	느림, 메모리 제어 어려움
Java	이식성, 안전성	느림, 저수준 제어 어려움
Rust	메모리 안전성	학습 난이도 높음, 생태계 작음
C++	성능 + 제어 + 생태계	학습 비용 있음

💡 "C++를 배우면 다른 언어가 왜 그렇게 설계되었는지 이해하게 됩니다."

C++ 프로그래밍

Junho Kim

Kookmin University

C++ 개발환경 구축

개발환경 개요

필요한 도구

```
1  소스 코드 (.cpp)
2      |
3      ▼
4  g++ (GCC C++ 컴파일러)
5      |
6      ▼
7  실행 파일 (a.out / .exe)
```

도구	역할
g++ (GCC)	C++ 컴파일러
GDB	GNU 디버거
VSCode	코드 편집기

개발환경 개요

OS별 설치 방법

OS	설치 방법
Windows	MSYS2 + MinGW-w64
macOS	Homebrew
Linux	apt / dnf (패키지 매니저)

💡 이 강의는 MSVC(Visual Studio)를 사용하지 않습니다.

크로스 플랫폼 환경(GCC + VSCode)에 집중합니다.

Windows

GCC (MinGW-w64) 설치

Windows — GCC 설치 (1/2)

Step 1 · MSYS2 설치

<https://www.msys2.org> 에서 설치 파일 직접 다운로드

Step 2 · MSYS2 UCRT64 터미널에서 GCC 설치

```
1  # MSYS2 UCRT64 터미널 실행 후 입력
2
3  pacman -Syu                      # 패키지 DB 업데이트
4
5  pacman -S --needed base-devel mingw-w64-ucrt-x86_64-toolchain # 개발용 툴체인 설치
```

Windows — GCC 설치 (2/2)

Step 3 · 환경변수 PATH 추가

- 1 [Windows 검색] → "시스템 환경 변수 편집"
- 2 → 환경 변수 → Path → 새로 만들기
- 3 → C:\msys64\ucrt64\bin 입력 후 확인

Step 4 · 설치 확인

```
1 g++ --version
2 # g++ (Rev1, Built by MSYS2 project) 13.x.x
3
4 gdb --version
5 # GNU gdb (GDB) 13.x.x
```

💡 MSYS2 터미널은 반드시 MSYS2 UCRT64 항목으로 실행하세요.
UCRT64 환경이어야 g++ 경로가 올바르게 잡힙니다.

macOS

GCC 설치 (Homebrew)

macOS — GCC 설치

macOS의 기본 상황 주의

```
1 gcc --version
2 # Apple clang version 15.x.x ← 실제로는 Apple Clang의 별칭(alias)!
```

macOS에서 `gcc` 는 Apple Clang의 별칭입니다. 진짜 GCC는 Homebrew로 설치합니다.

Step 1 · Homebrew 설치

```
1 /bin/bash -c "$(curl -fsSL \
2 https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Step 2 · GCC & GDB 설치

```
1 brew install gcc
2 brew install gdb
```

macOS — GCC 설치

Step 3 . 설치 확인 및 alias 등록

```
1 # Homebrew GCC는 버전 번호가 붙은 이름으로 설치됨
2 g++-13 --version
3 # g++ (Homebrew GCC 13.x.x) 13.x.x
```

버전 번호 없이 `g++` 로 사용하고 싶다면 `~/.zshrc` 에 alias 추가:

```
1 alias g++='g++-13'
2 alias gcc='gcc-13'
```

⚠ macOS에서 GDB는 **코드 서명(Code Signing)** 설정이 추가로 필요할 수 있습니다.
번거롭다면 VSCode의 LLDB 디버거를 대신 사용하세요.

Linux

GCC 설치 (apt / dnf)

Linux — GCC 설치

Ubuntu / Debian 계열 (apt)

```
1  sudo apt update
2  sudo apt install -y build-essential # gcc, g++, make 묶음 패키지
3  sudo apt install -y gdb
4
5  g++ --version
6  # g++ (Ubuntu 13.x.x) 13.x.x
```

Fedora / RHEL 계열 (dnf)

```
1  sudo dnf install -y gcc gcc-c++ gdb
2
3  g++ --version
```

💡 실습 환경으로 Ubuntu 22.04 LTS 사용을 권장합니다.

Linux — 여러 GCC 버전 관리

여러 버전 동시 설치 (Ubuntu)

```
1  sudo apt install -y g++-11 g++-12 g++-13
```

update-alternatives로 기본 버전 설정

```
1  sudo update-alternatives \
2    --install /usr/bin/g++ g++ /usr/bin/g++-13 100
3
4  sudo update-alternatives \
5    --install /usr/bin/g++ g++ /usr/bin/g++-12 90
6
7  # 대화형으로 기본 버전 선택
8  sudo update-alternatives --config g++
```

g++ 컴파일 옵션

& 첫 번째 빌드

첫 번째 빌드

hello.cpp 작성

```
1  #include <iostream>
2
3  int main() {
4      std::cout << "Hello, C++!" << std::endl;
5      return 0;
6  }
```

컴파일 & 실행

```
1  # 컴파일
2  g++ hello.cpp -o hello
3
4  # 실행
5  ./hello           # Linux / macOS
6  .\hello.exe       # Windows PowerShell
```

주요 g++ 컴파일 옵션

옵션	의미	사용 시점
<code>-std=c++17</code>	C++17 표준 사용	항상
<code>-Wall</code>	주요 경고 활성화	항상
<code>-Wextra</code>	추가 경고 활성화	항상
<code>-g</code>	디버그 정보 포함	개발 중
<code>-O0</code>	최적화 끄 (기본값)	디버깅 시
<code>-O2</code>	최적화 레벨 2	릴리즈 빌드
<code>-o <이름></code>	출력 파일명 지정	항상

권장 개발용 빌드 명령어

```
1 g++ -std=c++17 -Wall -Wextra -g -o hello hello.cpp
```

💡 `-Werror` 를 추가하면 경고를 오류로 처리합니다.

컴파일 4단계

```
1  # 1단계: 전처리 (.cpp → .i)
2  g++ -E hello.cpp -o hello.i
3
4  # 2단계: 컴파일 (.i → .s 어셈블리)
5  g++ -S hello.cpp -o hello.s
6
7  # 3단계: 어셈블 (.s → .o 오브젝트)
8  g++ -c hello.cpp -o hello.o
9
10 # 4단계: 링크 (.o → 실행파일)
11 g++ hello.o -o hello
```

```
1  .cpp →(전처리)→ .i →(컴파일)→ .s →(어셈블)→ .o →(링크)→ 실행파일
```

VSCode

설치 & C++ 환경 설정

VSCode 설치 & 확장

설치

OS	설치 방법
Windows	https://code.visualstudio.com 에서 다운로드
macOS	<code>brew install --cask visual-studio-code</code>
Ubuntu	<code>sudo snap install code --classic</code>

필수 확장(Extension) 설치

`Ctrl+Shift+X` → Extensions 탭

확장 이름	역할
C/C++ (Microsoft)	IntelliSense, 구문 강조, GDB 연동
C/C++ Extension Pack (Microsoft)	C/C++ + CMake Tools 묶음 (권장)

VSCode — c_cpp_properties.json

Ctrl+Shift+P → C/C++: Edit Configurations (UI) 선택

```
1  // .vscode/c_cpp_properties.json
2  {
3    "configurations": [
4      {
5        "name": "Linux",
6        "compilerPath": "/usr/bin/g++",
7        "cppStandard": "c++17",
8        "intelliSenseMode": "linux-gcc-x64"
9      }
10   ],
11   "version": 4
12 }
```

OS별 compilerPath

OS	경로
Windows	C:/msys64/ucrt64/bin/g++.exe
macOS	/opt/homebrew/bin/g++-13
Linux	/usr/bin/g++

VSCode — tasks.json

Ctrl+Shift+B 로 g++ 빌드를 자동 실행하는 설정 파일

```
1  // .vscode/tasks.json
2  {
3    "version": "2.0.0",
4    "tasks": [
5      {
6        "label": "Build with g++",
7        "type": "shell",
8        "command": "g++",
9        "args": [
10         "-std=c++17", "-Wall", "-Wextra", "-g",
11         "${file}",
12         "-o", "${fileDirname}/${fileBasenameNoExtension}"
13       ],
14        "group": { "kind": "build", "isDefault": true },
15        "problemMatcher": ["$gcc"]
16      }
17    ]
18  }
```

VSCode — tasks.json

변수	의미
<code>\${file}</code>	현재 열린 파일 전체 경로
<code>\${fileDirname}</code>	현재 파일의 디렉토리
<code>\${fileBasenameNoExtension}</code>	확장자 없는 파일명

VSCode — launch.json (GDB)

F5 키로 GDB 디버거를 실행하는 설정 파일

```
1  // .vscode/launch.json
2  {
3    "version": "0.2.0",
4    "configurations": [
5      {
6        "name": "Debug with GDB",
7        "type": "cppdbg",
8        "request": "launch",
9        "program": "${fileDirname}/${fileBasenameNoExtension}",
10       "stopAtEntry": false,
11       "cwd": "${fileDirname}",
12       "MIMode": "gdb",
13       "miDebuggerPath": "/usr/bin/gdb",
14       "preLaunchTask": "Build with g++"
15     }
16   ]
17 }
```

💡 preLaunchTask 의 값은 tasks.json의 label 과 정확히 일치해야 합니다.

VSCode — 디버그 단축키

단축키	동작
F5	디버그 시작 / 계속
F9	브레이크포인트 토글
F10	한 줄 실행 (Step Over)
F11	함수 내부로 진입 (Step Into)
Shift+F11	함수에서 빠져나오기 (Step Out)
Shift+F5	디버그 중지

OS별 miDebuggerPath

OS	경로
Linux	/usr/bin/gdb
macOS	/opt/homebrew/bin/gdb
Windows	C:/msys64/ucrt64/bin/gdb.exe

기본 Linux Shell 명령어

Shell 명령어 (1) — 파일 & 디렉토리

자세한 사항은 [missing-semester](#)를 방문해서 공부하도록 한다.

탐색 & 이동

```
1  pwd                # 현재 경로 출력
2  ls                 # 파일 목록
3  ls -al             # 숨김 파일 포함 상세 출력
4  cd ~/projects      # 디렉토리 이동
5  cd ..              # 상위 디렉토리로
6  cd -               # 이전 디렉토리로
```

생성 / 복사 / 삭제

```
1  mkdir my_project   # 디렉토리 생성
2  mkdir -p src/utils  # 중간 경로 포함 생성
3  touch main.cpp      # 빈 파일 생성
4  cp main.cpp backup.cpp # 파일 복사
5  cp -r src/ src_backup/ # 디렉토리 복사
6  mv main.cpp src/main.cpp # 이동 / 이름 변경
7  rm main.cpp         # 파일 삭제
8  rm -rf build/       # 디렉토리 삭제 ⚠ 복구 불가
```

Shell 명령어 (2) — 컴파일 & 검색

g++ 컴파일 패턴

```
1  g++ -std=c++17 -Wall -g main.cpp -o main && ./main
2
3  # 여러 파일
4  g++ -std=c++17 -Wall main.cpp utils.cpp -o app
5
6  # 오브젝트 파일 분리 빌드
7  g++ -std=c++17 -c main.cpp -o main.o
8  g++ -std=c++17 -c utils.cpp -o utils.o
9  g++ main.o utils.o -o app
```

검색 & 파이프

```
1  grep "main" main.cpp           # 문자열 검색
2  grep -rn "cout" ./src          # 재귀 검색 + 줄번호
3  find . -name "*.cpp"           # .cpp 파일 검색
4  find . -name "*.o" -delete     # .o 파일 전부 삭제
5
6  ls -al | grep ".cpp"           # 파이프: .cpp 파일만 필터
```

Shell 명령어 (3) — 편집기 & 권한

터미널 편집기

```
1 nano main.cpp      # 저장: Ctrl+O  /  종료: Ctrl+X
2
3 vim main.cpp
4 # i                → 입력 모드
5 # ESC              → 명령 모드
6 # :wq              → 저장 후 종료
7 # :q!              → 저장 없이 종료
```

파일 권한

```
1 ls -l main
2 # -rwxr-xr-x  (소유자 rwx / 그룹 r-x / 기타 r-x)
3
4 chmod +x main      # 실행 권한 추가
5 chmod 755 main      # rwxr-xr-x
6 chmod 644 main.cpp  # rw-r--r--
```

유용한 단축키

```
1 Ctrl+C  강제 종료  Ctrl+L  화면 지우기
2 Tab     자동완성   ↑/↓     히스토리 탐색
```


Shell 명령어 — 한눈에 보기

카테고리	핵심 명령어
탐색	<code>pwd</code> , <code>ls</code> , <code>cd</code>
파일 조작	<code>touch</code> , <code>cp</code> , <code>mv</code> , <code>rm</code> , <code>mkdir</code>
내용 확인	<code>cat</code> , <code>less</code> , <code>grep</code> , <code>find</code>
빌드 & 실행	<code>g++ -std=c++17 -Wall -g</code> , <code>./실행파일</code>
편집	<code>nano</code> , <code>vim</code>
권한	<code>chmod</code> , <code>ls -l</code>
기타	<code>which</code> , <code>man</code> , <code>history</code>

Q&A

C Programming 소개

Junho Kim

Kookmin University

C 프로그래밍 소개

C vs Python: 핵심 차이점

C vs Python: 핵심 차이점

Python과 C의 가장 큰 차이는 타입 선언, 컴파일, 메모리 관리입니다.

Python

```
1  # 타입 선언 불필요 (동적)
2  x = 10
3  name = "홍길동"
4
5  # 실행: python3 hello.py
6  print("Hello, World!")
7
8  # 메모리 자동 관리 (GC)
9  my_list = [1, 2, 3]
10
11 # 문자열 객체
12 s = "hello"
13 print(len(s)) # 5
```

C

```
1  // 타입 명시 필수 (정적)
2  int x = 10;
3  char name[] = "홍길동";
4
5  // 컴파일 후 실행
6  // gcc hello.c -o hello && ./hello
7  printf("Hello, World!\n");
8
9  // 메모리 수동 관리
10 int *arr = malloc(3 * sizeof(int));
11 free(arr);
12
13 // 문자열 = char 배열
14 char s[] = "hello";
15 printf("%zu\n", strlen(s)); // 5
```

C vs Python: 비교표

항목	Python	C
타입 선언	불필요 (동적)	필수 (정적)
실행 방식	인터프리터	컴파일 필요
메모리 관리	자동 (GC)	수동 malloc/free
문자열	<code>str</code> 객체	<code>char</code> 배열 + <code>\0</code>
표준 출력	<code>print()</code>	<code>printf()</code>
클래스/OOP	<code>class</code>	<code>struct</code> (메서드 없음)
문자열 비교	<code>s1 == s2</code>	<code>strcmp(s1, s2)</code>
배열 경계 검사	자동	없음 ⚠️

(이렇게 귀찮음에도) C를 배우는 이유

-  시스템 수준 프로그래밍 이해
-  메모리 구조 직접 제어
-  C++의 기반 — 오늘 배우는 것이 C++의 토대
-  운영체제, 임베디드, 게임 엔진의 언어

C 프로그래밍 소개

컴파일 & 빌드 과정

첫 번째 C 프로그램

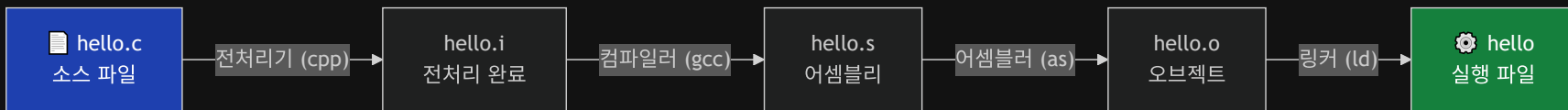
```
1  #include <stdio.h>    // 헤더 포함 (Python의 import)
2
3  int main(void) {      // 프로그램 진입점 (필수!)
4      printf("Hello, World!\n");
5      return 0;         // 0 = 정상 종료
6  }
```

컴파일 & 빌드 과정

```
1  # 기본 컴파일 & 링크 (한 번에)
2  gcc hello.c -o hello
3
4  # 권장 옵션 (경고 + C11 표준)
5  gcc -Wall -Wextra -std=c11 hello.c -o hello
6
7  # 실행
8  ./hello
```

자세한 컴파일 & 빌드 과정

소스 → 실행까지 4단계



주요 gcc 명령어

```
1  # 기본 컴파일 & 링크 (한 번에)
2  gcc hello.c -o hello
3
4  # 단계별 실행
5  gcc -E hello.c -o hello.i  # 전처리
6  gcc -S hello.c -o hello.s  # 어셈블리
7  gcc -c hello.c -o hello.o  # 오브젝트
8
9  # 권장 옵션 (경고 + C11 표준)
10 gcc -Wall -Wextra -std=c11 hello.c -o hello
11
12 # 실행
13 ./hello
```

C 프로그래밍 소개

변수, 자료형, 연산자

기본 자료형

정수형 & 문자형

```
1 // 정수형
2 char c = 'A'; // 1 byte (-128 ~ 127)
3 short s = 100; // 2 bytes
4 int i = 42; // 4 bytes ← 가장 많이 씀
5 long l = 1000000L; // 8 bytes
6
7 // 부호 없는 정수
8 unsigned int u = 4294967295U;
9
10 // 실수형
11 float f = 3.14f; // 4 bytes (단정밀도)
12 double d = 3.14159; // 8 bytes (배정밀도) ← 권장
13
14 // 논리값 (C99)
15 #include <stdbool.h>
16 bool flag = true; // 내부적으로 1/0
```

크기 확인 & printf 형식 지정자

```
1 printf("char : %zu bytes\n", sizeof(char));
2 printf("int : %zu bytes\n", sizeof(int));
3 printf("double : %zu bytes\n", sizeof(double));
```

형식 지정자	자료형	예시
%d	int	printf("%d", 42)
%f	double	printf("%.2f", 3.14)
%c	char	printf("%c", 'A')
%s	char	printf("%s", "hi")
%p	포인터	printf("%p", ptr)
%zu	size_t	printf("%zu", sizeof(x))

연산자

산술 연산자

```
1  int a = 10, b = 3;
2
3  a + b    // 13
4  a - b    // 7
5  a * b    // 30
6  a / b    // 3 ← 정수 나눗셈!
7  a % b    // 1 ← 나머지
8
9  // Python처럼 실수 나눗셈 하려면
10 (double)a / b // 3.333...
```

비교 & 논리

```
1  // 비교 (결과는 1 또는 0)
2  a == b    // 0 (false)
3  a != b    // 1 (true)
4  a > b     // 1 (true)
5  a >= b    // 1 (true)
6
7  // 논리 (Python: and, or, not)
8  a > 0 && b > 0 // and
9  a > 0 || b > 0 // or
10 !(a > 0)      // not
```

증감 & 대입

```
1  int x = 5;
2
3  x++;      // x = 6 (후위 증가)
4  ++x;      // x = 7 (전위 증가)
5  x--;      // x = 6 (후위 감소)
6
7  // 복합 대입
8  x += 3;   // x = x + 3
9  x -= 1;   // x = x - 1
10 x *= 2;   // x = x * 2
11 x /= 4;   // x = x / 4
12 x %= 3;   // x = x % 3
```

⚠ Python과 다른 점: `int / int` 는 항상 정수! `10 / 3 = 3` (나머지 버림)

C 프로그래밍 소개

조건문 & 반복문

조건문

조건식의 논리 연산, 블록 처리 방식 등이 다름

C

```
1 // 기본 if-else
2 int score = 85;
3
4 if (score >= 90) {
5     printf("학점: A\n");
6 } else if (score >= 80) {
7     printf("학점: B\n"); // ← 출력됨
8 } else {
9     printf("학점: F\n");
10 }
```

- 조건식에 괄호 `()` 필수
- 블록에 중괄호 `{}` 권장 (한 줄도!)
- 각 문장 끝에 세미콜론 `;` 필수

Python

```
1 # Python
2 if score >= 90:
3     print("학점: A")
4 elif score >= 80:
5     print("학점: B")
6 else:
7     print("학점: F")
```

- 조건식에 괄호 `()` 권장
- 블록은 들여쓰기(indentation) 필수
- 각 문장 끝에 세미콜론 `;` 필요 없음

반복문

for 루프

```
1 // 기본 for
2 for (int i = 0; i < 5; i++) {
3     printf("%d ", i);
4 }
5 // 출력: 0 1 2 3 4
6
7 // 중첩 for: 구구단
8 for (int i = 2; i <= 9; i++) {
9     for (int j = 1; j <= 9; j++) {
10        printf("%d*%d=%d\n",
11            i, j, i*j);
12    }
13 }
```

while / do-while

```
1 // while
2 int n = 1;
3 while (n <= 5) {
4     printf("%d ", n);
5     n++;
6 }
7 // 출력: 1 2 3 4 5
8
9 // do-while (최소 1회 실행)
10 int x;
11 do {
12     printf("양수 입력: ");
13     scanf("%d", &x);
14 } while (x <= 0);
```

break & continue

```
1 // break: 루프 탈출
2 for (int i = 0; i < 10; i++) {
3     if (i == 5) break;
4     printf("%d ", i);
5 }
6 // 출력: 0 1 2 3 4
7
8 // continue: 다음 반복으로
9 for (int i = 0; i < 10; i++) {
10    if (i % 2 == 0) continue;
11    printf("%d ", i);
12 }
13 // 출력: 1 3 5 7 9
```

```
1 // Python의 range(5)와 비교: for (int i=0; i<5; i++) 가 동일
2 // Python의 enumerate는 C에서 인덱스 변수 i로 직접 관리
```


C 프로그래밍 소개

함수 정의와 호출

함수의 선언, 정의, 호출

C 프로그램

```
1 // 1 프로토타입 선언 (declaration)
2 int add(int a, int b);
3 int factorial(int n);
4
5 // main 함수 - 프로그램 진입점
6 int main(void) {
7 // 2 함수 호출 (call)
8     int r = add(3, 5); // add() 함수 호출
9     printf("r: %d\n", r);
10    printf("5! = %d\n", factorial(5)); // 120
11
12    return 0;
13 }
14
15 // 3 함수 정의 (definition)
16 int add(int a, int b) { return a + b; }
17
18 int factorial(int n) { // 재귀
19     if (n <= 1) return 1;
20     return n * factorial(n - 1);
21 }
```

Python 프로그램

```
1 # Python - 타입 선언 불필요
2 def add(a, b):
3     return a + b
4
5 def factorial(n):
6     if n <= 1: return 1
7     return n * factorial(n - 1)
```

C 함수의 특징

- 반환 타입 명시 필수 (`int` , `void` , `double` ...)
- 프로토타입 선언 필요 (사용 전에 알려줘야 함)
- 함수 오버로딩 없음 → C++에서 지원
- 값에 의한 전달, call(pass)-by-value — 원본 변경 불가

```
1 // 함수 프로토타입 선언 (declaration)
2 int fun(int, int);
3
4 // 함수 호출 (call)
5 int main(void) {
6     int a = 3; int b = 5;
7     int r = fun(a, b);
8     printf("a = %d, b = %d, r = %d\n", a, b, r);
9     return 0;
10 }
11
12 // 함수 정의 (definition)
13 int fun(int a, int b) {
14     a = 1; b = 2;
15     return a + b;
16 }
```

C 언어의 함수 인자 전달: 값 전달 (call-by-value)

💡 C는 모든 것이 기본적으로 값 전달(call-by-value)입니다.

참조 전달(call-by-reference)가 가능한 Python/Java/C++ 등과 달리, C는 값 전달만 가능하므로 원본을 바꾸려면 **포인터**를 사용해야 합니다.

❌ call-by-value (원본 안 바뀜)

```
1 void swap_wrong(int a, int b) {
2     int tmp = a;
3     a = b;
4     b = tmp;
5     // 지역 복사본만 바뀜!
6 }
7
8 int main(void) {
9     int x = 10, y = 20;
10    swap_wrong(x, y);
11    printf("%d %d\n", x, y); // 10 20 - 그대로!
12 }
```

✅ 포인터 기반 call-by-value (원본 바뀜)

```
1 void swap(int *a, int *b) {
2     int tmp = *a;
3     *a = *b;
4     *b = tmp;
5     // 주소를 받아 원본 직접 수정
6 }
7
8 int main(void) {
9     int x = 10, y = 20;
10    swap(&x, &y); // 주소를 넘김
11    printf("%d %d\n", x, y); // 20 10 ✅
12 }
```

C 프로그래밍 소개

포인터 기초

포인터란?

💡 포인터(pointer)는 말 그대로 가리키는 것!

포인터 기반 코딩

```
1 // 변수: 값을 저장하는 공간
2 int x = 42;
3
4 // 포인터: 메모리 주소를 저장하는 변수
5 int *p = &x; // & = 주소 연산자
6 // *p 타입: int를 가리키는 포인터
7 // p에 x의 주소가 저장됨
8
9 // 값 출력
10 printf("x = %d\n", x); // 42
11 printf("&x = %p\n", &x); // 0x7fff... (주소)
12 printf("p = %p\n", p); // 0x7fff... (동일!)
13 printf("*p = %d\n", *p); // 42 (역참조)
14
15 // 포인터로 값 변경
16 *p = 100; // *p = 역참조 연산자
17 printf("x = %d\n", x); // 100 ← x가 바뀜!
18 // p를 통해 x를 직접 수정한 것
```

메모리 시각화

```
1          주소          값
2
3  ┌──────────┬──────────┐ ← int x
4  │ 0x1000   │ 42        │
5
6          ↑
7          │ p가 가리킴
8
9  ┌──────────┬──────────┐ ← int *p
10 │ 0x2000   │ 0x1000   │
11
12 *p = 100 → 0x1000 위치의 값을 100으로 변경
```

포인터 사용의 철칙

💡 포인터(pointer)는 말 그대로 가리키는 것!

- 이미 존재하는 것을 가리켜야 함
- 꼭 필요한 경우가 아니면 쓰지 말 것!!!
- 굳이 쓴다면 문법 구조를 이해하고 쓰자.

핵심 연산자 정리

연산자	이름	의미
<code>&x</code>	주소 연산자	x가 저장된 메모리 주소
<code>*p</code>	역참조 연산자	p가 가리키는 주소의 값
<code>int *p</code>	포인터 선언	p는 int를 가리키는 포인터

⚠ 포인터 관련 모든 문제는 포인터 연산의 직관적이지 못한 문법 구조 때문.

포인터 선언과 역참조 연산자의 표현이 동일... 이것이 모든 문제의 씨앗 😞

swap 구현 다시 보기: (call-by-value)

💡 C는 모든 것이 기본적으로 값 전달(call-by-value)입니다.

참조 전달(call-by-reference)가 가능한 Python/Java/C++ 등과 달리, C는 값 전달만 가능하므로 원본을 바꾸려면 **포인터**를 사용해야 합니다.

❌ call-by-value (원본 안 바뀜)

```
1 void swap_wrong(int a, int b) {
2     int tmp = a;
3     a = b;
4     b = tmp;
5     // 지역 복사본만 바뀜!
6 }
7
8 int main(void) {
9     int x = 10, y = 20;
10    swap_wrong(x, y);
11    printf("%d %d\n", x, y); // 10 20 - 그대로!
12 }
```

✅ 포인터 기반 call-by-value (원본 바뀜)

```
1 void swap(int* a, int* b) {
2     int tmp = *a;
3     *a = *b;
4     *b = tmp;
5     // 주소를 받아 원본 직접 수정
6 }
7
8 int main(void) {
9     int x = 10, y = 20;
10    swap(&x, &y); // 주소를 넘김
11    printf("%d %d\n", x, y); // 20 10 ✅
12 }
```


C 프로그래밍 소개

변수 Scope (유효 범위)

변수 Scope란?

변수가 선언된 위치에 따라 접근 가능한 범위가 결정됩니다.

```
1 // 전역 변수 (Global) - 파일 전체에서 접근 가능
2 int global = 100;
3
4 void funcA(void) {
5     // 지역 변수 (Local) - 이 함수 안에서만 유효
6     int local = 10;
7     global = 200; // ✅ 전역 변수 접근 가능
8     printf("%d\n", local); // 10
9     printf("%d\n", global); // 200
10 } // ← local은 여기서 소멸 (Stack에서 제거)
11
12 void funcB(void) {
13     // printf("%d\n", local); // ❌ 컴파일 오류!
14     printf("%d\n", global); // ✅ 200 (전역은 가능)
15 }
```

```
1 int main(void) {
2     int x = 42; // main의 지역 변수
3
4     { // 블록 scope
5         int y = 99;
6         printf("%d\n", x); // ✅ 42
7         printf("%d\n", y); // ✅ 99
8     }
9     // printf("%d\n", y); // ❌ y는 블록 밖에서 소멸!
10
11     funcA();
12     funcB();
13     return 0;
14 }
```

Scope 종류

Scope를 이해해야 포인터가 왜 필요한지 명확히 알 수 있습니다.

```
1 // 전역 변수 (Global) - 파일 전체에서 접근 가능
2 int global = 100;
3
4 void funcA(void) {
5     int local = 10;          // 지역 변수 (Local) - 이 함수 안에서만 유효
6     static int sss = 200;    // 정적 변수 (static) - 프로그램 종료까지
7
8     printf("%d\n", local);    // 10
9 } // ← local은 여기서 소멸 (Stack에서 제거)
10
11 int main(void) {
12     int x = 42;              // main의 지역 변수
13
14     { // 블록 scope
15         int y = 99;
16         printf("%d\n", y);    // ✅ 99
17     }
18     return 0;
19 }
```

Scope 종류 한눈에 보기

종류	선언 위치	유효 범위	수명
전역	함수 밖	파일 전체	프로그램 종료까지
지역	함수 안	해당 함수	함수 반환 시 소멸
블록	{ } 안	해당 블록	블록 종료 시 소멸
정적	static	지역과 동일	프로그램 종료까지

💡 변수의 수명(life-time)과 유효 범위(scope)을 혼동하지 말자!

swap 구현 다시 보기: (call-by-value)

💡 C는 모든 것이 기본적으로 값 전달(call-by-value)입니다.

참조 전달(call-by-reference)가 가능한 Python/Java/C++ 등과 달리, C는 값 전달만 가능하므로 원본을 바꾸려면 **포인터**를 사용해야 합니다.

❌ call-by-value (원본 안 바뀜)

```
1 void swap_wrong(int a, int b) {
2     int tmp = a;
3     a = b;
4     b = tmp;
5     // 지역 복사본만 바뀜!
6 }
7
8 int main(void) {
9     int x = 10, y = 20;
10    swap_wrong(x, y);
11    printf("%d %d\n", x, y); // 10 20 - 그대로!
12 }
```

✅ 포인터 기반 call-by-value (원본 바뀜)

```
1 void swap(int *a, int *b) {
2     int tmp = *a;
3     *a = *b;
4     *b = tmp;
5     // 주소를 받아 원본 직접 수정
6 }
7
8 int main(void) {
9     int x = 10, y = 20;
10    swap(&x, &y); // 주소를 넘김
11    printf("%d %d\n", x, y); // 20 10 ✅
12 }
```

포인터 사용의 철칙

코딩 규칙: 포인터 선언과 역참조 연산자의 표현을 구분해서 쓰자!!!

```
1  int* p; // 포인터 변수 선언
2  int a = 10;
3  p = &a;
4  printf("%d", *p); // 역참조 연산
```

코딩 규칙 (구분된 규칙)

연산자	이름	의미
<code>&x</code>	주소 연산자	x가 저장된 메모리 주소
<code>*p</code>	역참조 연산자	p가 가리키는 주소의 값
<code>int* p</code>	포인터 선언	p는 int를 가리키는 포인터

✓ 포인터 기반 call-by-value (원본 바뀜)

```
1  void swap(int* a, int* b) {
2      int tmp = *a;
3      *a = *b;
4      *b = tmp;
5      // 주소를 받아 원본 직접 수정
6  }
7
8  int main(void) {
9      int x = 10, y = 20;
10     swap(&x, &y); // 주소를 넘김
11     printf("%d %d\n", x, y); // 20 10 ✓
12 }
```